Array Programs in JAVA

1. Sum of Array

Logic:

- Start with sum = 0
- Add each element one by one
- Store result in sum

Code:

```
int arr[] = {10, 20, 30};
int sum = 0;

for(int i = 0; i < arr.length; i++) {
   sum = sum + arr[i];   // adding each element
}

System.out.println(sum);
```

Example flow:
0 + 10 = 10
10 + 20 = 30
30 + 30 = 60

2. Find Maximum Element

Logic:

- Assume first element is max
- Compare with all others
- Replace if bigger found

Code:

```
int arr[] = {5, 20, 15};

int max = arr[0];

for(int i = 1; i < arr.length; i++) {
   if(arr[i] > max) {
      max = arr[i];
   }
}

System.out.println("Max = " + max);
```

Logic line:

“Keep updating max when bigger value appears”

Reverse an Array (Using Swapping)

Code

```
public class ReverseArray {

  public static void main(String[] args) {

    int arr[] = {10, 20, 30, 40, 50};

    System.out.print("Original Array: ");
    for(int i = 0; i < arr.length; i++) {
      System.out.print(arr[i] + " ");
    }

    // Reversing using two-pointer technique
    int start = 0;
    int end = arr.length - 1;

    while(start < end) {

      // Swap elements
      int temp = arr[start];
      arr[start] = arr[end];
      arr[end] = temp;

      start++;
      end--;
    }

    System.out.print("\nReversed Array: ");
    for(int i = 0; i < arr.length; i++) {
      System.out.print(arr[i] + " ");
    }
  }
}
```

Logic ()

Step-by-step:

- Take two variables:
    - start = 0 (first element)
    - end = last index

Then:

1. Swap arr[start] and arr[end]
2. Move start++ (forward)
3. Move end-- (backward)
4. Repeat until they meet

What is Linear Search?

Linear Search means:
You check each element one by one until you find the target.

Simple words:
“Start from index 0 → go till end → compare each element”

Code

```
public class LinearSearch {

   public static void main(String[] args) {

      int arr[] = {10, 25, 30, 45, 50};
      int key = 30;

      int index = -1;   // default (not found)

     for(int i = 0; i < arr.length; i++) {

        if(arr[i] == key) {
           index = i;
           break;   // stop when found
        }
      }
      if(index != -1) {
        System.out.println("Element found at index: " + index);
      } else {
        System.out.println("Element not found");
      }

   }

}
```

Logic Explained

Idea:

- Compare each element with key

- If match → stop
- Else → keep checking

5. What is Sorting?

Sorting means arranging elements in order:

- Ascending → 1 2 3 4
- Descending → 4 3 2 1

Logic:

- Compare adjacent elements
- Swap if wrong order
- Repeat multiple times

Code:

```
int arr[] = {4,2,1,3};

for(int i = 0; i < arr.length; i++) {
   for(int j = 0; j < arr.length-1; j++) {
      if(arr[j] > arr[j+1]) {
         int temp = arr[j];
         arr[j] = arr[j+1];
         arr[j+1] = temp;
      }
   }
}

for(int num : arr) {
   System.out.print(num + " ");
}
```

Output: 1 2 3 4

1. PRINT MATRIX

Logic:

- Use 2 loops
    - Outer → rows
    - Inner → columns

Code:

```
public class PrintMatrix {
  public static void main(String[] args) {

    int arr[][] = {
      {1, 2, 3},
      {4, 5, 6}
    };

    for(int i = 0; i < arr.length; i++) {
      for(int j = 0; j < arr[i].length; j++) {
        System.out.print(arr[i][j] + " ");
      }
      System.out.println();
    }
  }
}
```

2. MATRIX ADDITION

Logic:

Add same position elements

c[i][j] = a[i][j] + b[i][j]

Code:

```
public class MatrixAddition {
   public static void main(String[] args) {

      int a[][] = {{1,2},{3,4}};
      int b[][] = {{5,6},{7,8}};
      int c[][] = new int[2][2];

      for(int i = 0; i < 2; i++) {
         for(int j = 0; j < 2; j++) {
            c[i][j] = a[i][j] + b[i][j];
            System.out.print(c[i][j] + " ");
         }
         System.out.println();
      }
   }
}
```

3. MATRIX MULTIPLICATION

Logic:

Row × Column multiplication

c[i][j] += a[i][k] * b[k][j]

Needs 3 loops

Code:

```
public class MatrixMultiplication {
   public static void main(String[] args) {

      int a[][] = {{1,2},{3,4}};
      int b[][] = {{5,6},{7,8}};
      int c[][] = new int[2][2];

      for(int i = 0; i < 2; i++) {
         for(int j = 0; j < 2; j++) {
            for(int k = 0; k < 2; k++) {
               c[i][j] += a[i][k] * b[k][j];
            }
            System.out.print(c[i][j] + " ");
         }
         System.out.println();
      }
   }
}
```

How it works (important!)

Example:

c[0][0] = (1×5) + (2×7)

Row of A × Column of B

4. TRANSPOSE MATRIX

Logic:

Convert rows into columns

arr[i][j] → arr[j][i]

Code:

```
public class TransposeMatrix {
  public static void main(String[] args) {

    int arr[][] = {
      {1,2,3},
      {4,5,6}
    };

    for(int i = 0; i < 3; i++) {
      for(int j = 0; j < 2; j++) {
        System.out.print(arr[j][i] + " ");
      }
      System.out.println();
    }
  }
}
```

FINAL UNDERSTANDING

Print Matrix → 2 loops
Addition → same index addition
Multiplication → 3 loops (row × column)
Transpose → swap indices

1. PRINT 3D ARRAY

Logic:

- 3 loops needed:
    - i → layer
    - j → row
    - k → column

Code:

```
public class Print3DArray {
  public static void main(String[] args) {

    int arr[][][] = {
      {{1,2},{3,4}},
      {{5,6},{7,8}}
    };

    for(int i = 0; i < arr.length; i++) {

      System.out.println("Layer " + i + ":");

      for(int j = 0; j < arr[i].length; j++) {

        for(int k = 0; k < arr[i][j].length; k++) {
          System.out.print(arr[i][j][k] + " ");
        }

        System.out.println();
      }
      System.out.println();
    }
  }
}
```

2. SUM OF 3D ARRAY

Logic:

Add all elements using 3 nested loops

Code:

```
public class Sum3DArray {
   public static void main(String[] args) {

      int arr[][][] = {
         {{1,2},{3,4}},
         {{5,6},{7,8}}
      };

      int sum = 0;

      for(int i = 0; i < arr.length; i++) {
         for(int j = 0; j < arr[i].length; j++) {
            for(int k = 0; k < arr[i][j].length; k++) {

               sum += arr[i][j][k];

            }
         }
      }

      System.out.println("Total Sum = " + sum);
   }
}
```

3. FIND MAX ELEMENT IN 3D ARRAY

Logic:

- Assume first element is max
- Compare all elements

```
int max = arr[0][0][0];

for(int i=0;i<arr.length;i++){
   for(int j=0;j<arr[i].length;j++){
      for(int k=0;k<arr[i][j].length;k++){
         if(arr[i][j][k] > max){
            max = arr[i][j][k];
         }
      }
   }
}

System.out.println("Max = " + max);
```

4. SEARCH ELEMENT IN 3D ARRAY

Logic:

Check every element

```
int key = 6;
boolean found = false;

for(int i=0;i<arr.length;i++){
   for(int j=0;j<arr[i].length;j++){
      for(int k=0;k<arr[i][j].length;k++){
         if(arr[i][j][k] == key){
            System.out.println("Found at: ["+i+"]["+j+"]["+k+"]");
            found = true;
         }
      }
   }
}

if(!found){
   System.out.println("Not Found");
}
```

5. COPY 3D ARRAY

Logic:

Copy element by element

```
int copy[][][] = new int[arr.length][arr[0].length][arr[0][0].length];

for(int i=0;i<arr.length;i++){
   for(int j=0;j<arr[i].length;j++){
      for(int k=0;k<arr[i][j].length;k++){
         copy[i][j][k] = arr[i][j][k];
      }
   }
}
```

6. COUNT EVEN & ODD IN 3D ARRAY

```
int even = 0, odd = 0;

for(int i=0;i<arr.length;i++){
   for(int j=0;j<arr[i].length;j++){
      for(int k=0;k<arr[i][j].length;k++){

         if(arr[i][j][k] % 2 == 0) even++;
         else odd++;

      }
   }
}

System.out.println("Even = " + even);
System.out.println("Odd = " + odd);
```

FINAL UNDERSTANDING

1D → 1 loop
2D → 2 loops
3D → 3 loops